MongoDB NoSQL server. One of the beauties of an ORM is that it abstracts this away; your code is the same regardless of what database is connected underneath.

When a Django view requires information out of the database it uses one or more ORM classes that have been defined by the programmer. Each class inherits from an ORM base class called `Model`. The base class provides methods for querying the database. For example, calling `.objects.filter(last_name__startswith="A")` on `Author` returns all the authors with a last name beginning with `"A"`. Query results get represented by a common Django object called a `QuerySet`. The `QuerySet` can be passed as part of a data context to the template engine. The template engine generates HTML using the results. A list of authors might generate a bullet-point list in the output. Django also provides a way to deal with the database directly if the abstraction is insufficient in specific cases.

1.1.3 Structuring HTML for reuse

Your Django view is responsible for returning content to the browser, which most of the time is HTML. HTML was originally designed for physicists to write easily accessible articles for other physicists. Modern web pages have more in common with marketing brochures than physics articles. Each page in a website tends to include a lot of repetition—for example, the navigation header that appears on every page.

Django includes a templating engine. The engine's job is to compose pieces of content together into a page. Using it, you only have to write the navigation header once, and then it can be included in every page of your site. To change the navigation header, you only have to do it in one place.

Your view likely wants to show some of that previously mentioned data on your page. Lists of things can get quite repetitive. In Python, if you want to print a long list, you don't hardcode it. Instead, you put a `print` statement inside a `for` loop that iterates on the list. The templating engine lets you do the same using *tags* and *filters*. Tags are instructions to the engine for conditional rendering, while filters change how data is displayed. Using these, you can create loops, conditionally render blocks, print a date in your user's format, and much more. Figure 1.8 shows how templates are composed and how to use tags, denoted by `{% %}`. All of this is covered in detail in the chapter on the template engine.

Django is able to interact with different template engines. The original templating language that ships with Django, understandably, is called the *Django Templating Language* (DTL). Django also ships with the popular third-party templating language Jinja2. This isn't maintained by the Django core team, and won't be covered in this book, but is included in the framework for the convenience of those familiar with it.

The DTL interface has three concepts: variable rendering, tags, and filters. Variable rendering replaces values in the template with data sent into the engine. Tags are like functions and provide looping, conditional blocks, and a number of operations on your templated text. Filters are variable modifiers changing values before they are displayed. For example, one filter takes a date object and formats it to a desired output. The DTL is also pluggable, allowing you to write your own tags and filters.